

Intelligent Guide v1 — Acceptance Walkthrough

End-to-end manual QA covering Sprints 3 + 4 + 5 + 6

Joe Pruskowski

2026-04-25

Contents

Run status — 2026-05-02 (local dev)	1
What this document covers	2
Stage 0 — Prereqs	2
Stage 1 — Phase 0 funnel (Sprints 1 + 2)	2
Stage 2 — Hook (Sprint 3)	3
Stage 3 — Curriculum: Quick Bot wizard (Sprint 3)	3
Stage 4 — Curriculum: Spar (Sprint 4)	4
Stage 5 — Curriculum: Cup (Sprint 4)	4
Stage 6 — Specialize: discovery rewards (Sprint 5)	5
Stage 7 — Admin metrics dashboard (Sprints 5 + 6)	5
Stage 8 — Admin SystemConfig editor (Sprint 6)	6
Stage 9 — CLI sanity (Sprint 5)	6
Stage 10 — Kill switch (Sprint 6)	7
Stage 11 — Concurrency + edge cases (post-2026-05-02 hardening)	7
Stage 12 — Research Log Sprint 1 (training session notes)	9
Sign-off	10

Run status — 2026-05-02 (local dev)

Stage	Description	Status
0	Prereqs (env up, services healthy)	not run
1	Phase 0 funnel (Sprints 1+2)	pending — manual
2	Hook (Sprint 3)	pending — manual
3	Curriculum: Quick Bot wizard (Sprint 3)	pending — manual
4	Curriculum: Spar (Sprint 4)	pending — manual
5	Curriculum: Cup (Sprint 4)	pending — manual
6	Specialize: discovery rewards (Sprint 5)	pending — manual
7	Admin metrics dashboard (Sprints 5+6)	pending — manual
8	SystemConfig admin panel (Sprint 6)	pending — manual
9	um admin commands (Sprint 6)	pending — manual
10	V1 release flag (Sprint 6)	pending — manual
11.1	Concurrent step-2 race (post-2026-05-02 hardening)	PASSED 2026-05-02 ☐
11.2	SystemConfig outage fail-closed (post-2026-05-02 hardening)	PASSED 2026-05-02 ☐
11.3	Out-of-order step credit / monotonicity (post-2026-05-02 hardening)	PASSED 2026-05-02 ☐

Stages 1–10 form the golden-path walkthrough and are scheduled for the next manual run on dev. Stage 11 covers the concurrency + edge-case hardening shipped on 2026-05-02 and is automated-friendly enough to re-run on demand. Once Stages 1–10 complete on dev, the same script runs against staging before production.

What this document covers

This is the single end-to-end QA script for the Intelligent Guide v1 release. It walks the *whole* journey — Phase 0 funnel, Hook, Curriculum, Specialize discovery rewards, the admin dashboard, the SystemConfig panel, and the kill switch — in one sitting, in chronological user order.

Run this on **local dev** before staging, and on **staging** before production. Each pass takes ~30 minutes if everything is green and nothing else is going on.

Companion docs:

- **What** — `Intelligent_Guide_Requirements.md`
- **How** — `Intelligent_Guide_Implementation_Plan.md`
- **Operations** — `Guide_Operations.md` (read this if a stage fails)
- **Acceptance (this doc)** — top-level pass/fail script for the v1 release

The earlier per-sprint QA docs (`Sprint3_QA.md`, `Sprint4_QA.md`) have been superseded by this walkthrough and moved to `/doc/archive/`. Refer to them only if you need historical detail on a specific sprint's deliverable.

If you're new to the guide work, read `Intelligent_Guide_Requirements.md` §3 (the phase model) and §4 (the 7 steps) first. This doc references them by section number throughout.

Stage 0 — Prereqs

1. Stack running — `docker compose ps` shows backend, landing, tournament, postgres, redis all healthy.
2. A clean browser session (incognito works) for the test user — most steps depend on a fresh signup.
3. A second browser tab signed in as an **admin** user for the admin-dashboard stages.
4. A terminal handy for `um` and `docker compose logs`.
5. `metrics.internalEmailDomains` should NOT include `@dev.local` for this run — we want the test user to count in the dashboard so we can see the funnel move.

If you're running this on staging, skip step 5 — staging keeps internal domains flagged and you'll be testing through real-customer view.

Stage 1 — Phase 0 funnel (Sprints 1 + 2)

The pre-signup flow on the landing page. The hero is the demo arena; below it sit the CTAs that funnel users into either PvAI (“Play”) or signup (“Build a bot”).

1.1 Fresh-load landing as a guest

- Open `http://localhost:5174/` in a clean session.
- Hero DemoArena renders: a bot-vs-bot match plays as the hero (no popup; the legacy `GuestWelcomeModal` was retired in `ec57188` — the demo arena IS the welcome).
- Three CTAs render directly under the arena: ☐ **Watch another match**, **Play against a bot**, **Build your own bot** →.

1.2 Play one PvAI

- Click “Play against a bot”. You land on the board, mark X by default.
- The page is intentionally minimal for the guest demo: just the table surface, the seat pods, and a small ← Back link. The platform sidebar (Game info, “Gym — train a bot”, “Back to Tables”) and its toggle are suppressed via `minimalChrome` — those concepts are introduced post-signup, and they competed with the conversion CTA below.

- Play through to a result (win, lose, or draw — doesn't matter).
- After a ~2-second pause (so the result pill gets its uncluttered moment), a narrow toast slides down from the top of the viewport (under the nav): "Like this? Save your progress." with a pulsing **Build your own bot** → button and a × dismiss. After another ~5 seconds the toast dims to 50% opacity (still clickable; hover restores). It re-attention-grabs on each new finished game. (You can also ← Back to home, where the same CTA sits in the hero ladder.)

1.3 Sign up

- Click "Build your own bot". Signup modal opens.
- Fill display name, email (v1ack+<rand>@dev.local), password (min 8).
- The 3-second anti-bot guard delays the submit button — wait it out.
- Click "Create account". Modal closes. You should land on the home page signed in.

1.4 Verify Hook step 1 was credited

- In the admin tab, run:
`um status v1ack+<rand>`
- You should see `journey.completedSteps` includes 1. `phase: hook`. (Step 1 is credited at PvAI completion *before* signup, then re-attributed via the guest-credit endpoint when the user signs up.)

Pass criteria: signup succeeds, no console errors, `um status` shows step 1 credited.

Stage 2 — Hook (Sprint 3)

Steps 2 of 7. The Hook closes with the +20 TC reward popup.

2.1 Welcome modal

- After signup, the welcome modal appears (one-time, per-user). Dismiss it.

2.2 Open a demo table

- The JourneyCard on the home page should show "Watch two bots battle" as the next step. Click it.
- A demo table opens; bot-vs-bot game starts within ~2 seconds.

2.3 Wait for the demo bot game to complete

- The match runs ~10–30 seconds at default pace. Wait for the result screen.
- A reward popup should appear: "+20 TC — welcome to the Arena."
- Dismiss the popup.

2.4 Verify step 2 + Hook reward

- Run `um status v1ack+<rand>` again.
- `completedSteps` should now include 1, 2.
- `phase` should be `curriculum`.
- `creditsTc` should be 20 (or +20 above whatever it was).

Pass criteria: popup rendered, phase flipped, +20 TC granted. If the popup didn't render but the credit went through, that's a partial pass — check `RewardPopup.test.jsx` covers the unit-test path; the popup may have auto-dismissed before you looked.

Stage 3 — Curriculum: Quick Bot wizard (Sprint 3)

Steps 3 + 4. Quick Bot creation + Quick Train.

3.1 Open the Quick Bot wizard

- The JourneyCard now shows “Create your first bot” (step 3). Click it.
- The Quick Bot wizard opens — three persona cards (Aggressive, Balanced, Defensive).
- Pick one (any). The wizard creates the bot via `POST /api/v1/bots/quick`.

3.2 Verify step 3 + bot tier

- After the wizard closes, you land on the bot’s profile page.
- Run `um status` — `completedSteps` should now include 1, 2, 3.
- The bot should be at the **novice** tier (Rusty-equivalent, per `guide.quickBot.defaultTier`).

3.3 Quick Train

- On the bot profile, click “**Quick Train**” (the prominent button).
- Wait for the train to finish — typically <2 seconds.
- The bot’s tier should flip from **novice** → **intermediate** (Copper-equivalent, per `guide.quickBot.firstTrainingTier`).

3.4 Verify step 4

- `um status` should now show steps 1, 2, 3, 4.
- Phase still `curriculum`.

Pass criteria: bot created at the configured tier, Quick Train flips the tier, step 4 credited.

Stage 4 — Curriculum: Spar (Sprint 4)

Step 5. Spar against a tier-graded opponent.

4.1 Click Spar

- On the bot profile, click “**Spar**” → pick “**easy**” opponent tier.
- A spar series starts. With 200ms/move and ≤9 moves per game, the whole match wraps in a few seconds.

4.2 Verify step 5

- After the spar completes, run `um status`.
- `completedSteps` should now include 1, 2, 3, 4, 5.
- Phase still `curriculum`.

Pass criteria: spar runs, step 5 credited within ~10 seconds of completion.

Stage 5 — Curriculum: Cup (Sprint 4)

Steps 6 + 7. Curriculum Cup clone + completion.

5.1 Click Curriculum Cup clone

- On the home page or bot profile, the JourneyCard now shows “Enter a tournament”. Click it.
- A new Curriculum Cup is cloned with your bot in slot 0 + 3 cup-clone opponents (4-bot bracket).

5.2 Verify step 6 fires immediately

- `um status` should show 1, 2, 3, 4, 5, 6 within ~10 seconds (the `participant:joined` publish is fire-and-forget; small grace window).

5.3 Wait for the cup to complete

- The cup runs 3 games sequentially ($R1 \times 2 + R2 \times 1$) at $\sim 1s/move$.
- Total ~ 30 seconds plus completion bookkeeping. Watch the cup page; the bracket fills in live.

5.4 Verify step 7 + Curriculum reward + coaching card

- After cup completion:
 - **Reward popup**: “+50 TC” (or whatever `guide.rewards.curriculumComplete` is set to).
 - **Coaching card**: depending on your bot’s `finalPosition`: `CHAMPION` (1st), `RUNNER_UP` (2nd), or `HEAVY_LOSS` (3rd or 4th). Read the card; dismiss when ready.
- `um status` should show all 7 steps complete.
- Phase should flip to `specialize`.
- `creditsTc` should be +50 above its post-Hook value.

Pass criteria: cup completes, +50 TC granted, coaching card matches `finalPosition`, `phase = specialize`.

Stage 6 — Specialize: discovery rewards (Sprint 5)

Two of the four discovery rewards have a real caller in v1; the other two ship as a future-proofing surface.

6.1 First non-default algorithm

- Go to ML Studio. Create a new bot with a **non-default algorithm** — `qLearning` or `DQN`.
- Train it (any session). On training completion, the discovery reward fires.
- Reward popup: “+10 TC — **first non-default algorithm**”.
- `um rewards show v1ack+<rand>` — should list `firstNonDefaultAlgorithm` as granted.

6.2 First real tournament win

- Enter a real tournament (NOT a Curriculum Cup — that’s excluded). Wait for it to complete with your bot in 1st place.
- (Easiest path: create a 2-bot tournament where the other slot is a weak system bot like Rusty.)
- Reward popup: “+25 TC — **first tournament win**”.
- `um rewards show` — `firstRealTournamentWin` granted.

6.3 Skipped (v1.1 surface only)

- `firstSpecializeAction` and `firstTemplateClone` have no production caller in v1. Don’t try to trigger them. Listed here so you know not to chase them.

Pass criteria: both real callers fire, popups render, `um rewards show` lists both grants. TC balance is `previous + 10 + 25`.

Stage 7 — Admin metrics dashboard (Sprints 5 + 6)

Switch to the admin browser tab.

7.1 Open `/admin/guide-metrics`

- North Star %, “X / Y eligible users” subtitle, trend chart.
- 7-step funnel — your test user should be visible at every step (the bars are absolute counts, not percentages).
- Signup-method split — `credential` vs `OAuth`.
- Footer: “Excluding N test users.”

7.2 Cohort granularity picker

- Switch the trend granularity dropdown from **Week** → **Day** → **Month**.
- The chart should re-render each time without errors.

- Day shows ~30 buckets (one per day in the 30-day window).
- Week shows ~5 buckets (ISO week starts).
- Month shows ~2 buckets (current + previous month, depending on date).

7.3 Visual sanity

- Bars should be the platform teal/blue. No empty / undefined / NaN labels.
- Trend chart should not be empty (assuming the metrics cron has been running).

Pass criteria: dashboard renders, all four panels populate, granularity picker re-renders without errors.

Stage 8 — Admin SystemConfig editor (Sprint 6)

/admin → scroll to the “Intelligent Guide v1” panel.

8.1 Read current values

- All 13 keys load with their defaults: `guide.v1.enabled = true`, `guide.rewards.hookComplete = 20`, `guide.cup.sizeEntrants = 4` (read-only, disabled), etc.

8.2 Tune a reward → verify next user gets the new value

- Change `guide.rewards.hookComplete` from 20 → 25. Click Save → “☐ Saved”.
- Sign up a *new* test user (`v1ack-tune+<rand>@dev.local`).
- Walk them through Stage 1 + Stage 2 (PvAI + demo table watch).
- After step 2 fires, the reward popup should read “+25 TC”.
- `um status` for the new user — `creditsTc` is 25, not 20.
- Reset `guide.rewards.hookComplete` back to 20 in the panel.

8.3 Read-only key

- Try to edit `guide.cup.sizeEntrants` — the input should be disabled with a “v1.1” hint. The admin UI prevents the write; if you bypass the UI and PATCH directly, the backend rejects with “is read-only in v1”.

8.4 Internal email domains

- Add `dev.local` to `metrics.internalEmailDomains` (comma-separated textarea).
- Save. Run `um testuser --audit` — your existing `v1ack+...@dev.local` accounts should now appear as drift candidates (flagged in the domain list, but `isTestUser=false` on the user row).
- Run `um testuser --apply` to reconcile. Audit should come back clean.
- Remove `dev.local` from the list, save again. (Don’t leave dev domains in production config.)

Pass criteria: values load, edits persist, read-only enforced, domain changes drive audit + apply correctly.

Stage 9 — CLI sanity (Sprint 5)

A 2-minute confidence check that the operator tooling matches the dashboard.

9.1 `um status`

```
um status v1ack+<rand>
```

Expected fields: `username`, `email`, `isTestUser` (boolean), `journey.phase`, `journey.completedSteps`, `creditsTc`, `discoveryGrants`.

9.2 um testuser --audit

```
um testuser --audit
```

Should list any users where `User.isTestUser` disagrees with the `metrics.internalEmailDomains` rule. Empty output = perfectly reconciled.

9.3 um rewards show

```
um rewards show v1ack+<rand>
```

Should list each discovery reward key with `granted: true|false` and `grantedAt`.

Pass criteria: all three commands return clean structured output, no stack traces.

Stage 10 — Kill switch (Sprint 6)

The release-gate flag. Verify both directions: off silently no-ops credits, on resumes them.

10.1 Disable the guide

- Admin UI → Intelligent Guide v1 → uncheck `guide.v1.enabled`.
- Browser confirm dialog explains the cost — accept.
- Click Save.

10.2 Verify credits no-op

- Sign up another fresh test user (`v1ack-flag+<rand>@dev.local`).
- Walk them through Stage 1.4 (PvAI to end).
- Run `um status v1ack-flag+<rand>` — `journey.completedSteps` should be **empty** (`[]`). `Phase` = hook (default for no progress).
- The PvAI was played, the game record exists, but no journey credit fired — confirming the flag.

10.3 Re-enable the guide

- Admin UI → check `guide.v1.enabled` → Save (no confirm dialog when re-enabling).

10.4 Verify credits resume

- For the same `v1ack-flag+<rand>` user, play *another* PvAI to end.
- `um status` should now show step 1 credited (the new PvAI fired the trigger; the earlier one is lost — intentional).

Pass criteria: off-window credits are dropped (not deferred), on-state immediately resumes credits for new actions.

Stage 11 — Concurrency + edge cases (post-2026-05-02 hardening)

These checks pin two specific defects that were fixed alongside the journey-test hardening pass on 2026-05-02. They aren't part of the normal user flow, but they're cheap to run and they catch the class of regression that's hardest to spot from the dashboard (silent double-payment, propagated config errors).

11.1 Concurrent step-2 credit pays Hook reward exactly once — PASSED 2026-05-02

What this guards: before the fix, two simultaneous credits for the same step (multi-tab, or two backend pods reacting to the same Redis event) both passed the dedup check and both fired `_handleHookComplete` — paying +40 TC instead of +20. The fix wraps the read-modify-write in a per-user `pg_advisory_xact_lock` transaction (backend/src/services/journeyService.js).

Steps:

1. Sign up a fresh user (`v1ack-race+<rand>@dev.local`). Confirm `um status` shows `completedSteps: []`.
2. Play **one** PvAI game to completion (any opponent, any result). This credits step 1, leaving step 2 as the next-pending step. Confirm `um status` now shows `completedSteps: [1], phase: hook`.
3. In **tab A**, open the journey panel and click “**Watch a demo**”. The page redirects to `/tables/<demoId>` — note that URL.
4. In **tab B** (same browser, same logged-in session), paste the exact `/tables/<demoId>` URL. Do **not** click “Watch a demo” in tab B — that would mint a *different* demo table and you’d lose the race condition. Both tabs must be spectating the same `tableId`.
5. Let both tabs watch ≥ 2 minutes. Each tab runs its own 120s spectate timer; when both fire, they POST the step-2 credit concurrently — this is the race window the advisory lock guards.
6. After both tabs have rendered the +20 TC popup (or after a fresh page load on either tab), check:
 - `um status v1ack-race+<rand>` → `journey.completedSteps` includes 2 exactly once (no duplicates in the array).
 - User’s TC balance should be +20, not +40. Read it from the profile dropdown or `um wallet show v1ack-race+<rand>`.
 - Backend logs (`docker compose logs backend | grep "Hook complete"`) show the reward grant logged exactly once for this user.

Pass criteria: balance shows +20 TC, not +40. The race is closed.

Result (2026-05-02): PASSED. User `qqq` showed `creditsTc=20`, `completedSteps=[1,2]`, single “Hook complete — TC awarded” log line. Advisory lock confirmed working under real two-tab race.

11.2 SystemConfig outage fails closed (no propagated 500) — PASSED 2026-05-02

What this guards: the `guide.v1.enabled` lookup used to live outside the try/catch — a SystemConfig DB failure propagated the error up to every step-credit call site (game-end, tournament bridge, etc.), turning a single config-table hiccup into a cascade. The fix moved the lookup inside the existing try/catch; failures now return false silently.

Steps:

1. Pick a user mid-curriculum (`qqq` works, or any `v1ack-flag+<rand>` from Stage 10). Note their `creditsTc` and `journeyProgress` baseline.
2. Simulate the outage by **renaming the system_config table** so Prisma reads error immediately:

```
docker compose exec -T postgres psql -U xo -d xo_arena -c \
  "ALTER TABLE system_config RENAME TO _system_config_outage_test;"
```

(We use `rename` rather than `LOCK TABLE` because Postgres has no `statement_timeout` set — a lock would *hang* the read indefinitely instead of erroring, which doesn’t exercise the catch path. `Rename` produces an instant `relation does not exist` error, which is the failure mode the fix actually defends against.)

3. With the table renamed, have the user trigger any step-credit code path (play a PvAI game to end is easiest — even though step 1 is already credited, the trigger still calls `completeStep` and exercises the SystemConfig lookup before the dedup check).
4. Check the backend response and logs:
 - Game POST should return **200**, not 500.
 - Backend logs show a `WARN` line: `Journey step completion failed (non-fatal)` with the `err` containing `relation "system_config" does not exist` (or similar).
 - No unhandled-promise-rejection trace, no 500-level response anywhere in the network tab.

5. Restore the table:

```
docker compose exec -T postgres psql -U xo -d xo_arena -c \
  "ALTER TABLE _system_config_outage_test RENAME TO system_config;"
```

6. Have the user fire a step-trigger event that produces a *new* credit (e.g., create a bot to credit step 3 if they’re at step 2). `um status` should show the new step credited and TC balance updated.

Pass criteria: during the outage, the call site stays clean (200 response, warn log only, no cascading 500s). After the rename is restored, credits resume on the next trigger.

Result (2026-05-02): PASSED. Outage simulated by renaming `system_config` for ~54 seconds; user `qqq` played a PvAI game during the window with no 500s. Backend logs showed exactly one `Journey step completion failed (non-fatal)`

warn (with the underlying relation "system_config" does not exist Prisma error), no unhandled-promise rejections, no stack-trace propagation. Bonus finding: creditService and the table-GC sweep also fail closed under the same outage — the defensive pattern is consistent across services. After table rename was reversed, qqg created a bot and step 3 credited cleanly (completedSteps=[1,2,3]).

11.3 Out-of-order step credit (monotonicity policy spot-check) — PASSED 2026-05-02

What this guards: documented decision (Implementation Plan §8) — the service layer allows out-of-order completion, the only invariant is dedup. Step 7 alone fires the +50 TC reward and Specialize transition without prior steps, so a user who somehow lands on step 7 first still graduates cleanly. (The natural flow is steered by UI — this only matters for admin tools or future direct API access.)

Steps:

1. Pick a fresh user with journey.completedSteps == [].
2. Run `um journey complete-step <user> 7` (or equivalent admin tool) — bypass the natural triggers.
3. Verify:
 - `um status <user>` shows completedSteps: [7] and phase: specialize.
 - User's TC balance went up by +50 (Curriculum reward fires regardless of prior state).
 - SSE events `guide:curriculum_complete` and `guide:specialize_start` were emitted (visible in browser if user is online; otherwise check Redis stream).

Pass criteria: non-monotonic credit produces the same reward + transition as a complete journey. Confirms the documented policy is wired correctly.

Note on tooling: `um journey complete-step` doesn't exist as a CLI subcommand — `um journey <user> 0000001` would write completedSteps: [7] directly into preferences but skip the reward + SSE emission paths, defeating the test. Run instead:

```
docker compose exec -T backend node --experimental-transform-types --no-warnings --input-type=module
↳ -e \
  "import('./src/services/journeyService.js').then(async ({ completeStep }) => { await
  ↳ completeStep('<userId>', 7); await new Promise(r => setTimeout(r, 1000)); process.exit(0); })"
```

The 1-second sleep is needed because `appendToStream` is fire-and-forget — without it, the process exits before Redis `XADD` completes and the SSE events aren't written.

Result (2026-05-02): PASSED. Fresh user `v1ack-mono` (`cmooohdw9m...`, `creditsTc=0`, `completedSteps=[]`) had `completeStep(userId, 7)` invoked directly. Post-call: `creditsTc=50`, `completedSteps=[7]`, `phase=specialize`. All three expected SSE events landed on the Redis events stream: `guide:journeyStep` (step=7, phase=specialize), `guide:curriculum_complete` (reward=50), and `guide:specialize_start`. Monotonicity policy confirmed — service layer enforces dedup only, never prior-step ordering.

Stage 12 — Research Log Sprint 1 (training session notes)

Purpose: verify the user can write, edit, and delete notes attached to a training session, and that the routes correctly enforce ownership. Sprint 1 is publish-free — every note is private; publish + community lanes ship in Sprint 2.

Plan reference: `doc/Research_Log_Plan.md §3 “Sprint 1”`.

12.1 Add a note to one of your own sessions

Sign in as a user who has at least one completed `TrainingSession`. Open the Gym, select the model that owns the session, switch to the **Sessions** tab, and click a session row to open its detail panel. Expand the ☐ **Research notes** drawer. Type a body, choose an outcome (SUCCESS etc.), add one or two tags, and click + **Add note**.

Expected: the new note appears at the top of the list (newest first by default) with the outcome badge + tags rendered. The textarea clears.

12.2 Edit + delete the note

Click **Edit** on the new row, change the body and outcome, click **Save**. The row updates inline. Click **Delete** — the row disappears.

Expected: both mutations are optimistic; a transient network failure rolls back to the previous state and surfaces the error inline.

12.3 Ownership gate (cross-user 403)

In a separate browser session, sign in as a different user and call:

```
curl -X POST https://<host>/api/v1/research/sessions/<other-user-session-id>/notes \
  -H "Authorization: Bearer <your-token>" \
  -H "Content-Type: application/json" \
  -d '{"body":"x","outcome":"SUCCESS"}
```

Expected: 403 forbidden. No row is created. The Sessions tab of the other user's Gym is not even reachable in the UI, so this only matters as a direct API check.

12.4 Persistence on reload

Reload the page. Re-open the Gym, the same model, the same session row, the notes drawer.

Expected: every note created in 12.1 / 12.2 still renders.

12.5 Sort toggle

Click the **Newest first** ↓ button in the drawer header.

Expected: order flips to oldest-first; the button label flips to **Oldest first** ↑. Click again to revert.

Sign-off

If every stage passed, the v1 release is acceptance-clean. Tag the build and proceed to staging.

If any stage failed, file a bug referencing the stage number. The runbook (Guide_Operations.md §1.2) lists likely causes for each failure mode; the archived per-sprint QA docs in /doc/archive/ (Sprint3_QA.md / Sprint4_QA.md) have deeper repro detail if you need it.

Auto-coverage: the e2e/tests/guide-onboarding.spec.js Playwright spec walks Stages 1–5 in a single user flow as a smoke check. It does not cover Stages 6–10 (those need either a real ML training step, real tournaments, or admin-UI keystrokes — all manual). Run the E2E first; if it's green, this manual walkthrough is the confidence check that the dashboard + admin surfaces work too.

```
cd e2e && npm run test:e2e -- guide-onboarding.spec.js
```